A Field Guide to 3B-Class LLMs on the Jetson Orin Nano 8 GB

Setup, flags, benchmark suite, and everything we learned along the way

Walker Kirkpatrick Hermes (Nous Research)

October 11–12, 2026

Abstract

This guide documents, end to end, how we run small local language models as a resident agent backend on a Jetson Orin Nano 8 GB: building llama.cpp for the Orin, choosing and downloading models, every launch flag we use and why, our standard 34-prompt benchmark suite, and the honest findings from testing Qwen3.5-2B, LFM2.5-VL-3B and VibeThinker-3B at 64K context. It also records the operational scars — OOM kills, cgroup traps, a server-port collision — so you can avoid repeating them. Everything here was measured on the real board, not read off a datasheet.

Contents

1 Why this guide exists **3**

2 The hardware and software stack **3**
2.1 The one hardware fact that explains everything 3

3 Install: building llama.cpp for the Orin **3**
3.1 Prerequisites 3
3.2 Build 3
3.3 Downloading models 4

4 The launch line, flag by flag **4**
4.1 Environment variables 5
4.2 Model and serving flags 5
4.3 Sampling and thinking control (server-side defaults) 6
4.4 Speculative decoding: the speed lever 6
4.5 Context extension (YaRN) — only when the model needs it 6

5 Keeping it alive: the keepalive cron **6**

6 The standard benchmark suite **7**
6.1 The cap story (or: why v3 exists) 7
6.2 Methodology rules we do not bend 7

7 Findings I: the models we tested **8**
7.1 Qwen3.5-2B — the incumbent (and why it deserves the job) 8
7.2 LFM2.5-VL-3B — the fast, frugal, YARN-stretched challenger 8
7.3 VibeThinker-3B — the honest plodder with a natively huge window 9
7.4 The deep needle: what 64K actually means 9

8 Findings II: the 3B-class landscape from GGUF headers **9**

9 Findings III: the operations scars **10**
9.1 The terminal background wrapper is a memory trap 10
9.2 Two runners cannot share port 8090 10
9.3 The OOM killer has a favourite victim 10

10 Recommendations **11**

1 Why this guide exists

We run an AI agent (Hermes) whose brain lives entirely on a small ARM board with 8 GB of shared CPU/GPU memory. Every model that wants the job has to fit that budget *while the agent gateway, browser, and tooling are also resident*, pass a tool-calling test on the first turn, answer reasoning traps honestly, and be fast enough that a conversation doesn't feel geological.

In October 2026 we ran a systematic bake-off: three candidate models through an identical 34-prompt suite, one fresh server load per test, with tokens/second recorded on every single prompt. This document is both the manual (how to reproduce the setup) and the lab notebook (what we found, including the failures). The tone is deliberately direct — these were real experiments with real surprises, and the surprises are the useful part.

2 The hardware and software stack

Component	Detail
Board	Jetson Orin Nano 8 GB (Tegra234, device tree "Orin (nvgpu)"; unified memory — CPU and GPU share the same 8 GB)
GPU compute capability	8.7 (Ampere-class iGPU) — the value that goes into the CUDA build
CUDA toolchain	CUDA 12.6 (`/usr/local/cuda-12.6/bin/nvcc`)
Inference engine	llama.cpp, Release build, in `~/llama.cpp/build/bin/` (server binary: `llama-server`)
Resident agent	Hermes; gateway + browser sessions live in the same 8 GB
Production model	Qwen3.5-2B Q4_K_M at 64K context, q8_0 KV cache, embedded MTP speculative decoding, served at `http://127.0.0.1:8090/v1`
Watchdog	cron job pings the health endpoint every minute and relaunches if the server is down (`llama-keepalive.sh`)
GUI	disabled (gdm off) — headless saves roughly half a gigabyte

2.1 The one hardware fact that explains everything

The Orin Nano has *unified* memory: there is no separate VRAM. A model's weights, its KV cache, the CUDA context, the agent gateway, and Chromium all compete for the same 8 GB. A model that "fits" in isolation can still destabilise the system once the gateway is resident. That is why this guide quotes *free-after-load* numbers rather than model file sizes, and why several decisions below look conservative until you remember that the board is also running an agent.

3 Install: building llama.cpp for the Orin

3.1 Prerequisites

- JetPack with CUDA 12.6 installed and `nvcc` on PATH.
- Standard build toolchain: `cmake`, `build-essential`, `libcurl4-openssl-dev` (for the model downloader).
- Power profile unlocked: `sudo jetson_clocks` (our launch script calls it automatically; clocks drop otherwise and every tok/s number in this guide assumes it).

3.2 Build

```
sudo apt install -y build-essential cmake libcurl4-openssl-dev
git clone https://github.com/ggml-org/llama.cpp ~/llama.cpp
cmake -B ~/llama.cpp/build -S ~/llama.cpp \
```

```
    -DCMAKE_BUILD_TYPE=Release \
    -DGGML_CUDA=ON \
    -DCMAKE_CUDA_ARCHITECTURES=87 \
    -DCMAKE_CUDA_COMPILER=/usr/local/cuda-12.6/bin/nvcc
cmake --build ~/llama.cpp/build --config Release -j4
```

The two Orin-specific values are the compute architecture (87) and the CUDA compiler path. On a Jetson, always set `CMAKE_CUDA_ARCHITECTURES` explicitly — a wrong or missing arch produces PTX-only binaries that run, but far slower than they should, and the symptom is easy to misattribute to the model.

3.3 Downloading models

GGUF quantizations go to a models directory; we keep ours at `~/models/new-zoo/`. Two reliable routes:

```
# Route 1: llama.cpp's own downloader (knows the right quant names)
~/llama.cpp/build/bin/llama-cli -hf <org>/<repo>:Q4_K_M

# Route 2: explicit download (any HF mirror of GGUF files)
huggingface-cli download <org>/<repo> --include "*.Q4_K_M.gguf" \
    --local-dir ~/models/new-zoo
```

Always verify what you actually downloaded. Native context length, layer count, and KV geometry live in the GGUF header, and the values printed on a model card are not always what's in the file. A 30-line Python script (ours is `gguf_ctx.py` in the LFM benchmark repo) reads the header directly and settles any argument. Two examples where this mattered: our Qwen3.5-2B is natively 256K context (not the 32K we assumed), and the "48K-token" needle prompt turned out to be only 30.6K tokens on LFM's tokenizer — more on both below.

4 The launch line, flag by flag

This is the production launch line for the resident Qwen3.5-2B model, as it lives in `~/bin/llm-64k` today:

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 GGML_CUDA_NO_PINNED=1 \
setsid ~/llama.cpp/build/bin/llama-server \
    -m ~/models/new-zoo/Qwen_Qwen3.5-2B-Q4_K_M.gguf \
    --alias qwen3.5-2b --host 127.0.0.1 --port 8090 \
    -ngl 99 -c 65536 -ctk q8_0 -ctv q8_0 \
    -b 512 -ub 512 -fa on --jinja \
    --spec-type draft-mtp --spec-draft-n-max 3 \
    -np 1 -t 6 \
    --temp 0.0 --top-k 1 --repeat-penalty 1.0 \
    --chat-template-kwargs '{"enable_thinking": false}' \
    --fit off \
    > /tmp/llm-64k.log 2>&1 &
```

4.1 Environment variables

Variable	What it does, and why on this board
`GGML_CUDA_ENABLE_UNIFIED_MEMORY=1`	Lets the CUDA backend allocate from the board's unified pool normally. On Tegra this is the cooperative setting; without it, large allocations can fail early rather than degrade.
`GGML_CUDA_NO_PINNED=1`	Disables pinned (page-locked) host memory. On unified-memory Jetsons pinned allocations double-count against the pool and can trigger OOM kills during prompt processing; disabling them measurably reduces kill frequency, with no speed loss in our runs.

4.2 Model and serving flags

Flag	Meaning and our reasoning
`-m <file>`	Path to the GGUF. Obvious, but note we always pass the absolute path — relative paths break when the server is launched by a script from an unexpected working directory.
`-alias <name>`	The model name clients see in the API. Our agent config points at the alias, so swapping the underlying file does not require touching the agent.
`-host 127.0.0.1 -port 8090`	Loopback only. The server has no authentication; binding it to anything else is an open LLM endpoint on your LAN.
`-ngl 99`	Offload 99 layers to GPU. On an 8 GB Orin with a 2–3 GB model, every layer goes to the iGPU; a CPU fallback would be an order of magnitude slower.
`-c 65536`	Total context window: 64K tokens. This is the number the whole project revolves around — see §**??** for why 64K is the sweet spot on this board.
`-ctk q8_0 -ctv q8_0`	Quantize the KV cache to 8-bit (both keys and values). For Qwen3.5-2B this halves KV memory versus f16 and is the difference between fitting with the gateway resident or not. Quality impact at q8_0 was unmeasurable on our suite. For models with hybrid/cheap KV (LFM2.5-VL-3B) we omit these flags and run f16 KV instead — cheaper than it sounds, see §7.2.
`-b 512 -ub 512`	Physical and logical batch size for prompt processing. 512 is a good balance on this board; bigger batches spike unified memory during prefill.
`-fa on`	Flash attention. Materially reduces memory traffic during attention and is effectively free speed on Orin.
`-jinja`	Use the model's embedded Jinja chat template. Critical for tool calling: the template knows how to render `<tool_call>` markup and stop tokens correctly. Without it, several of our models silently fell back to a generic ChatML that broke native tool calls.
`-fit off`	Do not compress or substitute the model's KV dtype behind our backs when memory is tight; we want a loud failure, not a quiet quality drop.
`-np 1`	One parallel slot. Every extra slot reserves another full context worth of compute buffer; with one agent consumer, one slot is right.
`-t 6`	Six CPU threads for the non-GPU portions (tokenization, sampling glue). On the Orin Nano's 6-core CPU this leaves headroom for the gateway.

4.3 Sampling and thinking control (server-side defaults)

Flag	Meaning and our reasoning
`-temp 0.0 -top-k 1`	Greedy decoding as the server default. The agent sets its own sampling per request; these server-side values are a safety net so a bare curl gets deterministic behavior. (Our benchmark harness deliberately overrides this per request — see §6.)
`-repeat-penalty 1.0`	Repetition penalty off. Small models with strong meta-reasoning habits loop *harder* when penalized; we saw worse text, not better, at 1.1.
`-chat-template-kwargs`	Passes template variables server-side. For Qwen3.5 this is where `enable_thinking: false` lives: the model would otherwise spend its budget emitting hidden thinking tokens that the agent never sees and pays for anyway. This one line was worth several points of perceived quality.

4.4 Speculative decoding: the speed lever

Flag	Meaning and our reasoning
`-spec-type draft-mtp`	Multi-token-prediction self-drafting: the model's own embedded MTP heads propose 3–4 token continuations that the main pass verifies at once. No separate draft model to load — on an 8 GB board that is the whole point. This took Qwen3.5-2B from the high-20s to 37 tok/s average, peaking at 46.6.
`-spec-draft-n-max 3`	Cap drafted tokens per step. Beyond 3 the acceptance rate on our workloads dropped and the gain flattened.

Not every model has MTP heads, and the flag is model-specific in practice. LFM2.5-VL-3B pairs with a small external drafter (DSpark) invoked with `-model-draft` instead; VibeThinker-3B has neither, and it shows — a flat 20 tok/s in every single test (see §7.3).

4.5 Context extension (YaRN) — only when the model needs it

Flag	Meaning and our reasoning
`-rope-scaling yarn` `-rope-scale 2.0`	Stretches RoPE position frequencies to extend a model past its trained context (here 32K to 64K). It works mechanically — the server loads, attention doesn't explode — but *trained depth is a quality cliff, not a wall.* LFM2.5-VL-3B recalled our needle exactly at 30.6K and silently dropped a suffix at 49.3K. If a model needs YaRN to reach your working context, deep-recall is now on your test plan (§7.4).

The clean alternative is a model that is *natively* long-context. Our GGUF header survey (§8) found that Qwen3.5-2B is natively 256K — no YaRN, no cliff — which is a big part of why it holds the production slot.

5 Keeping it alive: the keepalive cron

```
# crontab -l (the one line that matters)
* * * * * /home/walker/.local/bin/llama-keepalive.sh \
    >> /home/walker/.local/bin/llama-keepalive.log 2>&1
```

The script pings `http://127.0.0.1:8090/health`; on failure it relaunches via `~/bin/llm-64k start`. It has genuinely resurrected the server in production within a minute of an OOM kill.

Two hard-won caveats, both of which we hit in practice:

- **It resurrects servers you meant to stop.** During any benchmark campaign you must comment the line out first (`crontab -e`), or it will relaunch the production model between your test loads and steal the port.
- **It papers over the OOM pattern.** Our llama-server runs at `oom_score_adj=0`, which makes it a favourite target of the kernel OOM killer whenever memory spikes (it was killed twice in one day until we understood why). The cron kept it alive, which is exactly the right behavior — but the real fix is lowering `OOMScoreAdjust` on the service, which is still on our list.

6 The standard benchmark suite

Every model gets the same 34 prompts, verbatim, in four suites — defined once in `prompts_abcd.json` so results stay comparable across models and across time. The canonical copy lives in the LFM benchmark repo, and each new model project copies it untouched.

Suite	Theme	Contents	Max tokens (v3)
A — general	everyday quality	one-shot code, iambic pentameter poem, clinical prose, creative scene, a $\sqrt{2}$ irrationality proof	6500
B — one-shot coding	can it ship code	HTML/CSS page, Python data-processing class, C system program, TRS-80 BASIC retro game, Julia numerics	24000
C — domain & tools	knowledge + function calls	zoo/ornithology HTML, Python, poem, an induction proof, a story, and six native tool-call tests (web search, terminal, file write/read, SQL, email)	4096 (1024 for tool calls)
D — reasoning traps	honesty under pressure	letter counting (strawberry, elephant), sheep arithmetic, bat & ball, sum 1–100, $\sqrt{2}$ proof, light switches, mislabeled boxes, binary search, Fibonacci, piano tuners, ice floating	3072

6.1 The cap story (or: why v3 exists)

The caps above are the current defaults, but they were not the first guess. The original suite ran at 400–1024 tokens, and it systematically punished verbose models: truncation, not capability, drove the scores. Each revision was driven by data:

Profile	Caps (A / B / C / D)	What forced the change
v1 (original)	6000 / 2000 / 2048–512 / 1024	Historical baseline; comparable to the pre-bake-off table of 20 models.
v2	6500 / 4000 / 4096–1024 / 3072	Qwen suite-A jumped 5.4→7.8 at higher caps: truncation artifact, not capability. VibeThinker truncated 6/12 of suite D at 1024.
v3 (current)	6500 / **24000** / 4096–1024 / 3072	VibeThinker hit 2000 *and* 4000 on the same B prompts; the deep budget finds where completion actually happens (b_html finished at 6,326 clean). Trim once measured.

6.2 Methodology rules we do not bend

1. **One fresh server load per test.** Kill, launch, wait for health, run exactly one prompt, kill. This isolates every test from cache state and measures honest load times. (For quick

non-comparative smoke tests a shared server is fine.)

2. **Sampling:** temperature 0.3, top_p 0.95, top_k 40, min_p 0.0 — matching the original suite runs so scores are comparable. Greedy (temp 0) is reserved for needle retests, where it distinguishes sampling noise from real degradation.
3. **Streaming metrics per prompt:** TTFT, prompt-processing tok/s, generation tok/s, effective tok/s, finish reason, and full usage — appended incrementally to a JSONL file so a crash never loses data and a rerun resumes at the first missing prompt.
4. **Execute the code.** Any prompt marked `exec_check` gets its generated code *run*, not read. This rule exists because Qwen's CSV class ran cleanly end-to-end while dividing department sums by the wrong denominator — a bug every prior inspection-based grade had missed. Grades cite execution output.
5. **Grade 1–10 against a written rubric** stored with each prompt in the suite file, and record tok/s next to the grade — the speed/quality trade-off is the entire point of the exercise.

7 Findings I: the models we tested

7.1 Qwen3.5-2B — the incumbent (and why it deserves the job)

The production model. Q4_K_M (1.7 GB), 64K context with q8_0 KV, MTP drafting, thinking disabled server-side. Verified: RSS 3–4 GB with 3 GB free and the gateway resident.

Metric	Result
Generation speed	37 tok/s average, 46.6 peak (MTP active)
Overall suite grade	7.4/10 at v1 caps; suite A alone rose 5.4→7.8 when uncapped to 6K
Tool calls	6/6 correct on the first turn — the only model to manage that
Hard reasoning traps	4/4 (strawberry, bat & ball, sheep, sum 1–100)
Proofs	$\sqrt{2}$ and induction proofs both 10/10 given room to run
Weak spots	multi-step spatial puzzles (light switches 2/10, mislabeled boxes 3/10), retro TRS-80 BASIC dialect, and iambic pentameter — it writes confident, consistently tetrameter verse at any budget
Style	verbose markdown/LaTeX; truncates under 1K-token caps (that was a budget artifact, not a capability ceiling)
Native context	256K — the 64K we run is *inside* its trained window. No YaRN, no cliff.

The 64K needle at depth was exact — which we later realised was never a surprise: the model was trained beyond the depth we were testing.

7.2 LFM2.5-VL-3B — the fast, frugal, YARN-stretched challenger

Paired with the DSpark speculative drafter, f16 KV, and YaRN rope-scale 2.0 to reach 64K from its native 32K. Its hybrid KV costs about 20 KB/token, so 64K of f16 KV is only about 0.6–1.25 GB — the cheapest 64K on the board.

Metric	Result
Generation speed	35.2 tok/s average, 51.7 peak; one-shot coding suite 44.6
Token economy	36% fewer tokens than Qwen on identical prompts; 3 truncations vs Qwen's 7; finished the two coding prompts Qwen ran out of budget on
Reasoning traps	5/5 correct, with cleaner breakdowns than Qwen's
Tool calls	5/6 — all first-turn and well-formed; the miss was a SQLite query emitted in *psql* syntax (right idea, wrong CLI)
Server cycles	34 loads, avg 9.4 s, RSS 4.84 GB, minimum free 1,349 MB — comfortable
Deep recall	the finding of the campaign; see §7.4

7.3 VibeThinker-3B — the honest plodder with a natively huge window

Native 128K context (2× headroom at 64K, no YaRN), Q4_K_M 1.93 GB, ChatML template with native tool calls — and, despite the name, *no thinking switch in its template.* Its reasoning is emitted as visible content, and that fact shapes everything about its results.

Metric	Result
Generation speed	20.3 tok/s, flat as a ruler, in every single test — no drafter exists for this model
Memory	RSS 4.75 GB at 64K f16 KV; loads in ∼8 s; 1.5–1.7 GB free after load
Reasoning traps	5/5 correct, at any cap
Suite A (6500)	4/5 clean stops; code 1,633 tok (grade 8), prose 5,877 (7), creative 4,024 (6), math 2,870 with a boxed proof (9); suite average 6.4
The poetry failure	iambic pentameter is a *hard loop*: all 6,500 tokens spent drafting and re-counting syllables; no poem was ever emitted at any budget. Unlike Qwen/LFM (who write a poem and break the meter), it never commits to verse at all
Suite D (1024)	6/12 truncated — its think-aloud style burns budget mid-thought
Suite D (3072)	3/12 truncated; the three rescued prompts completed with full reasoning; switches/boxes/piano loop past even 3K — the same classes Qwen fails, but VibeThinker spends triple the tokens trying
Suite B	3/3 truncated at 2000; b_html and b_python truncated again at 4000; at 24,000 b_html completed cleanly at 6,326 tokens (∼5.3 min) — the deep-cap probe doing its job

The recurring pattern: honest reasoning, correct short-answer traps, real (if verbose) deliverables — and a 3–10× token overhead versus Qwen/LFM because the meta-planning ("We need to…") is the content. At 20 tok/s that overhead is also a wall-clock overhead.

7.4 The deep needle: what 64K actually means

The needle test plants a fact at the very start of a long prompt — "The secret access word for the vault is PINECONE-42." — buries it under thousands of lines of filler, and asks for it at the end. It is the only test in our arsenal that separates *having* a context window from being able to *use* one.

Depth (real)	vs. native 32K	Sampling	Answer
30,640 tok	inside	temp 0.3	PINECONE-42 (exact)
49,340 tok	17K beyond	temp 0.3	PINECONE — the "-42" dropped
49,340 tok	17K beyond	greedy	PINECONE — deterministic: real, not noise

Method note, because it bit us: the filler that yields 48K tokens on Qwen-class tokenizers is only 30.6K on LFM's — tokenizers differ per model, and the real depth must always be read back from the API's `usage.prompt_tokens`, never estimated. That is how our first "48K" test turned out to be an in-training-depth control group.

The verdict for LFM2.5-VL-3B: YaRN stretches *semantics* but not *exact recall.* Past its trained depth it retrieves the meaning of the needle and loses the suffix. Fine for summarising a long document; not fine for recalling an ID, a code, or a verbatim string from the far end of the window. Gemma-4-E2B, for reference, passed this same needle at 48K — YaRN outcomes are per-model, which is why the needle is now mandatory for any context-extended candidate.

8 Findings II: the 3B-class landscape from GGUF headers

We read native context and KV geometry directly out of every GGUF on disk (no model loads needed). This table settles "which 3B can really do 64K" without a single benchmark run:

Model (all on disk)	Native ctx	64K KV (q8)	Verdict on this board
Qwen3.5-2B (production)	256K	1.68 GB	already there — no YaRN needed
SmolLM3-3B	64K	∼2.4 GB	true native 64K, but its tool calls don't parse — disqualified for the agent slot
Granite-4.2-3B	128K	hybrid (cheap)	tested: 3/4, only 4.6 tok/s — too slow
VibeThinker-3B	128K	∼1.2 GB	cheapest native fit; see §7.3
Ministral-3 / Shieldstral-1.0 (3B)	256K	3.49 GB	tight but plausible; untested
Nanbeige4.2-3B	256K	2.95 GB	marginal, and 11 tok/s looped-compute tax
Qwen3.5-4B	256K	4.43 GB	native yes, but KV + weights + gateway exceeds 7 GB — doesn't fit
Qwen2.5-3B	32K	—	only fully-verified 3B at 64K q8 (4/4, 15.7 tok/s), but needs YaRN above 32K
LFM2.5-VL-3B	32K	—	fastest (with DSpark), but YaRN recall cliff applies

The honest state of the intersection [native 64K] ∩ [fits 8 GB with gateway] ∩ [agent-capable]: Qwen3.5-2B is the only verified occupant. Native support alone is not the binding constraint — memory is.

9 Findings III: the operations scars

Every rule below was paid for in lost runs or dead servers. All three were journal-verified, not theorised.

9.1 The terminal background wrapper is a memory trap

The agent terminal's background mode wraps commands in a systemd scope (`hermes-worker-*.scope`) whose memory cgroup is too small for a model load. llama-server reached ∼4.3 GB RSS and the cgroup OOM killer executed it, every time — two full benchmark batches died to this before we read the journal and saw the pattern.

The rule: model loads never run through the background wrapper. Long batches are launched *detached* — a `Popen` with `start_new_session=True` issued from a normal foreground call — which lands the process in the roomy user slice. Progress is then read from the log file and the JSONL row count; the harness is resume-safe, so even a hard kill loses nothing but time. (Shell-level `setsid/nohup &` is refused by the terminal tool anyway; the in-process variant is the sanctioned route.)

9.2 Two runners cannot share port 8090

We once launched a 24K batch while a 4K-cap runner was still mid-flight. Their kill/launch cycles raced on the port; the new batch lost every single launch (five "SERVER LOAD FAILED" rows) and its sibling recorded a corrupted row with no usage.

The rule: the harness now takes a lockfile (`/tmp/vibe_bench.lock`, `O_EXCL`, with stale-lock stealing so a SIGKILLed holder can't block forever) and a second runner aborts cleanly instead of racing. One runner on the port, ever.

9.3 The OOM killer has a favourite victim

llama-server at `oom_score_adj=0` is the fattest process on the board, so any global memory spike (Chromium, the gateway) tends to get resolved by killing the model. The keepalive cron brings it back within a minute — good for uptime, bad for attribution, since it can resurrect a server you

meant to stop for a campaign. Pause the cron line, run the campaign, restore it. The durable fix — `OOMScoreAdjust=-500` on the service — remains on the list.

10 Recommendations

- **Resident agent slot: keep Qwen3.5-2B.** Fastest with MTP, only model with 6/6 first-turn tool calls, native 256K so 64K is in-training depth, and 3 GB free with the gateway resident.
- **When you need throughput on long code:** LFM2.5-VL-3B + DSpark is the best pure speed on the board (44.6 tok/s on coding, 36% fewer tokens) — as long as the working set stays inside its native 32K. Beyond that, expect semantic-but-not-exact recall.
- **When you need a big honest window and can wait:** VibeThinker-3B gives 128K natively, honest traps 5/5 and real deliverables — at 20 tok/s and a 3–10× token tax. Give it generous caps (it completes; it just talks first) and keep it away from strict-form poetry.
- **For any context-extended model:** the deep needle is mandatory, the depth must be read from `usage` not estimated, and a failure deserves a greedy rerun before you believe it.

Colophon

All numbers in this guide were measured on the board itself between 2026-10-10 and 2026-10-12, with the benchmark harnesses and raw JSONL results preserved in the project folders `qwen35-2b-benchmark`, `lfm25-3b-64k-benchmark`, and `vibethinker-3b-128k-benchmark` under `~/projects`. Suite B at 24,000 max tokens was still completing as this document was written; b_html's clean 6,326-token finish is included, and the remaining rows will join the same JSONL. The LaTeX source of this guide lives beside it.